COMPILER DESIGN LAB

Programs, Sample Outputs & SLR Parsing Tables

Prepared from the source files uploaded in this conversation

Included: C programs, Lex/Flex programs, sample outputs, and ACTION/GOTO tables for both SLR programs.

Important note: C outputs were checked using sample inputs where practical. Flex/Lex is not installed in this environment, so outputs for the .l files are illustrative and based on the supplied examples/intended rules. The .l files are reproduced as uploaded, including any formatting issues present in the originals.

Contents

1. Heap memory allocation — heap.c
2. Keyword and identifier checking — keywords.c
3. Number to words — lex100.l
4. Keyword/identifier recognition — lexanalyser.l
5. Comment handling — lexcomment.l
6. Real number to integer part — lexconvert.l
7. Days in a month — lexmonth.l
8. Integer and real number classification — lexreal.l
9. Predictive parser — predictive.c
10. Recursive-descent expression parser — recursive.c
11. Recursive-descent list parser — recursive2.c
12. SLR parser for expressions — slr1.c (ACTION/GOTO tables)
13. SLR parser for a/b grammar — slr2.c (ACTION/GOTO tables)
14. Dynamic stack — stack.c

1. Heap memory allocation (heap.c)

Demonstrates First Fit, Best Fit, and Worst Fit memory allocation. Sample uses First Fit.

Source code (reproduced from uploaded file)

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int nb, np, i, j, choice, index;

    printf("Enter number of memory blocks: ");
    scanf("%d", &nb);

    if (nb <= 0)
    {
        printf("Invalid number of blocks.\n");
        return 1;
    }

    int *block = malloc(nb * sizeof(int));

    if (block == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    printf("Enter sizes of memory blocks:\n");
    for (i = 0; i < nb; i++)
        scanf("%d", &block[i]);

    printf("Enter number of processes: ");
    scanf("%d", &np);

    if (np <= 0)
    {
        printf("Invalid number of processes.\n");
        free(block);
        return 1;
    }

    int *process = malloc(np * sizeof(int));
    int *allocation = malloc(np * sizeof(int));

    if (process == NULL || allocation == NULL)
    {
        printf("Memory allocation failed.\n");
        free(block);
        free(process);
        free(allocation);
        return 1;
    }

    printf("Enter sizes of processes:\n");
    for (i = 0; i < np; i++)
    {
        scanf("%d", &process[i]);
        allocation[i] = -1;
    }

    printf("\n1. First Fit\n");
    printf("2. Best Fit\n");
    printf("3. Worst Fit\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    if (choice < 1 || choice > 3)
    {
        printf("Invalid choice.\n");
        free(block);
        free(process);
        free(allocation);
        return 1;
    }

    for (i = 0; i < np; i++)
    {
        index = -1;

        for (j = 0; j < nb; j++)
        {
            if (block[j] >= process[i])
            {
                if (choice == 1)
                {
                    index = j;
                    break;
```

```
                }
                else if (choice == 2)
                {
                    if (index == -1 || block[j] < block[index])
                        index = j;
                }
                else if (choice == 3)
                {
                    if (index == -1 || block[j] > block[index])
                        index = j;
                }
            }
        }

        if (index != -1)
        {
            allocation[i] = index;
            block[index] -= process[i];
        }
    }

    printf("\nProcess Size\tBlock Allocated\n");

    for (i = 0; i < np; i++)
    {
        printf("%d\t\t", process[i]);

        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }

    printf("\nRemaining memory in each block:\n");

    for (i = 0; i < nb; i++)
        printf("Block %d: %d\n", i + 1, block[i]);

    free(block);
    free(process);
    free(allocation);

    return 0;
}
```

Sample output

```
Enter number of memory blocks: 5
Enter sizes of memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of processes:
212 417 112 426

1. First Fit
2. Best Fit
3. Worst Fit
Enter choice: 1

Process Size    Block Allocated
212             2
417             5
112             2
426             Not Allocated

Remaining memory in each block:
Block 1: 100
Block 2: 176
Block 3: 200
Block 4: 300
Block 5: 183
```

2. Keyword and identifier checking (keywords.c)

Checks whether a word is a listed flow-control keyword or a valid C-style identifier.

Source code (reproduced from uploaded file)

```
#include <stdio.h>

int main()
{
    char str[50];
    const char *keywords[] = {
        "if", "else", "while", "do", "switch",
        "case", "default", "continue", "return",
        "goto", "for", "break"};
    int n = sizeof(keywords) / sizeof(keywords[0]);
    int i, j, found = 0;
    printf("Enter a word: ");
    scanf("%49s", str);
    for (i = 0; i < n; i++)
    {
        j = 0;
        while (str[j] == keywords[i][j] && str[j] != '\0')
            j++;
        if (str[j] == '\0' && keywords[i][j] == '\0')
        {
            found = 1;
            break;
        }
    }

    if (found)
    {
        printf("%s is a flow control keyword\n", str);
    }
    else
    {
        if ((str[0] >= 'A' && str[0] <= 'Z') ||
            (str[0] >= 'a' && str[0] <= 'z') ||
            (str[0] == '_'))
        {

            found = 1;

            for (i = 1; str[i] != '\0'; i++)
            {
                if (!((str[i] >= 'A' && str[i] <= 'Z') ||
                      (str[i] >= 'a' && str[i] <= 'z') ||
                      (str[i] >= '0' && str[i] <= '9') ||
                      (str[i] == '_')))
                {
                    found = 0;
                    break;
                }
            }

            if (found)
                printf("\n%s is a valid identifier", str);
            else
                printf("\n%s is not a valid identifier", str);
        }
        else
        {
            printf("\n%s is not a valid identifier", str);
        }
    }

    return 0;
}
```

Sample output

```
Enter a word: for
for is a flow control keyword

Another test:
Enter a word: total_2

total_2 is a valid identifier
```

3. Number to words (lex100.l)

Lex rules intended to convert selected numbers into English words.

Source code (reproduced from uploaded file)

```
%{
    #include <stdio.h>
    void units();
%}

% %
20 / [0 - 9] printf("Twenty");
30 / [0 - 9] printf("Thirty");
40 / [0 - 9] printf("Forty");
50 / [0 - 9] printf("Fifty");
60 / [0 - 9] printf("Sixty");
70 / [0 - 9] printf("Seventy");
80 / [0 - 9] printf("Eighty");
90 / [0 - 9] printf("Ninety");

10 printf("Ten");
11 printf("Eleven");
12 printf("Twelve");
13 printf("Thirteen");
15 printf("Fifteen");

1 [46789]
{
    units();
    printf("teen");
}

[1 - 9] units();
0 printf("Zero");
% %

void units()
{
    switch (yytext[yyleng - 1]) {
        case '1':
            printf("One");
            break;
        case '2':
            printf("Two");
            break;
        case '3':
            printf("Three");
            break;
        case '4':
            printf("Four");
            break;

        case '5':
            printf("Five");
            break;

        case '6':
            printf("Six");
            break;

        case '7':
            printf("Seven");
            break;

        case '8':
            printf("Eight");
            break;

        case '9':
            printf("Nine");
            break;
    }
}

int main()
{
    yylex();
    return 0;
}

int yywrap()
{
    return 1;
}
```

Sample output

```
Input: 25
```

```
Output: TwentyFive

Input: 42
Output: FortyTwo

Note: illustrative expected output based on the intended Lex rules.
```

Lex/Flex note: Flex/Lex was not available to execute these source files here. The output is illustrative, based on the accompanying examples and intended rules; it is not a verified run. The source listing itself is kept as uploaded.

4. Keyword/identifier recognition (lexanalyser.l)

Recognizes selected keyword spellings and identifier-like tokens.

Source code (reproduced from uploaded file)

```
%{
    #include <stdio.h>
%}
%%
If|else|while|do|switch|case|break|for|default
    {
        printf("Keyword");
    }
IF|ELSE|WHILE|DO|SWITCH|CASE|BREAK|FOR|DEFAULT
    {
        printf("Keyword");
    }
[A-Za-z]+[A-Za-z0-9_]*
    {
        printf("Identifier");
    }
%%
int main()
{
    yylex();
    return 0;
}
int yywrap()
{
    return 1;
}

Compilation:
lex lexanalysis.l
        cc lex.yy.c -
    ll
             ./                                                  .
        a.out
            If
                Keyword
                    FOR
                        Keyword
                            Abc123_def identifier
```

Sample output

```
Input: If FOR Abc123_def
Output: Keyword Keyword Identifier

Note: spaces may be echoed by Lex's default rule; exact formatting depends on the scanner rules and
input.
```

Lex/Flex note: Flex/Lex was not available to execute these source files here. The output is illustrative, based on the accompanying examples and intended rules; it is not a verified run. The source listing itself is kept as uploaded.

5. Comment handling (lexcomment.l)

Lex rules intended to detect comment text and replace ordinary text.

Source code (reproduced from uploaded file)

```
%{
    #include <stdio.h>
%}

%%
[/][/][a-zA-Z0-9]*
    {
        printf("%s", yytext);
    }

[a-zA-Z0-9]*
    {
        printf(" ");
    }

[/][*][a-zA-Z0-9]*[*][/]
    {
        printf("%s", yytext);
    }
%%

int main()
{
    yylex();
    return 0;
}
int yywrap()
{
    return 1;
}
Compilation:lex comments.l
            cc lex.yy.c -ll
            ./a.out
            Hello // world
            world
            hai /*friend*/
            friend
```

Sample output

```
Sample input (as noted with the source):
Hello // world
world
hai /*friend*/
friend

Expected intent: print comment text and replace ordinary words with spaces.
Note: the uploaded patterns may not match all comment forms as intended.
```

Lex/Flex note: Flex/Lex was not available to execute these source files here. The output is illustrative, based on the accompanying examples and intended rules; it is not a verified run. The source listing itself is kept as uploaded.

6. Real number to integer part (lexconvert.l)

Prints characters before the decimal point and exits.

Source code (reproduced from uploaded file)

```
%{
    #include <stdio.h>
    #include <stdlib.h>
    int i, j;
%}

%%
[0-9]*\.[0-9]+
    {
        for (i = 0; i < 10; i++)
        {
            if (yytext[i] == '.')
            {
                for (j = 0; j <= i - 1; j++)
                {
                    printf("%c", yytext[j]);
                }
            }
        }
        exit(0);
    }
%%

int main()
{
    yylex();
    return 0;
}

int yywrap()
{
    return 1;
}

Compilation:lex realtoint.l
            cc lex.yy.c -ll
            ./a.out
            24.12
            12
```

Sample output

```
Input: 24.12
Output: 24

The code prints the characters before the decimal point and exits.
```

Lex/Flex note: Flex/Lex was not available to execute these source files here. The output is illustrative, based on the accompanying examples and intended rules; it is not a verified run. The source listing itself is kept as uploaded.

7. Days in a month (lexmonth.l)

Prints month length; for February, asks for a year.

Source code (reproduced from uploaded file)

```
%{
    #include <stdio.h>
    int year;
    void leap();
%}

%%
jan|mar|may|july|aug|oct|dec
    {
        printf("31 days");
    }

april|june|sep|nov
    {
        printf("30 days");
    }

feb
    {
        leap();
    }

[a-zA-Z]*
    {
        printf("invalid");
    }
%%

int main()
{
    yylex();
    return 0;
}

void leap()
{
    printf("Enter year: ");
    scanf("%d", &year);

    if (year % 4 == 0)
        printf("29 days");
    else
        printf("28 days");
}

int yywrap()
{
    return 1;
}

Compilation:lex daysinamonth.l
            cc lex.yy.c -ll
            ./a.out
            jan
            31 days
            june
            30 days
            feb
            enter year
            1984
            29 days
```

Sample output

```
Input: jan
Output: 31 days

Input: june
Output: 30 days

Input: feb
Enter year: 1984
Output: 29 days
```

Lex/Flex note: Flex/Lex was not available to execute these source files here. The output is illustrative, based on the accompanying examples and intended rules; it is not a verified run. The source listing itself is kept as uploaded.

8. Integer and real number classification (lexreal.l)

Lex rules intended to classify positive/negative integers and real numbers.

Source code (reproduced from uploaded file)

```
% {
    #include <stdio.h>
% }

% %
"+"?[0-9]+
    {
        printf("%s: positive integers", yytext);
    }

-[0-9]+
    {
        printf("%s: negative integers", yytext);
    }

-[0-9]+\.[0-9]+
    {
        printf("%s: negative real numbers", yytext);
    }

"+"?[0-9]+\.[0-9]+
{
    printf("%s: positive real numbers", yytext);
}
% %

    int main()
{
    yylex();
    return 0;
}

int yywrap()
{
    return 1;
}

Compilation:lex noformat.l
            cc lex.yy.c -ll
            ./a.out
            24 positive integer
            +24.12 positive real number
            -24 negative integer
            -24.12 negative real number
```

Sample output

```
Input: 24
Output: 24: positive integers

Input: +24.12
Output: +24.12: positive real numbers

Input: -24
Output: -24: negative integers

Input: -24.12
Output: -24.12: negative real numbers

Note: intended output from the comments accompanying the source.
```

Lex/Flex note: Flex/Lex was not available to execute these source files here. The output is illustrative, based on the accompanying examples and intended rules; it is not a verified run. The source listing itself is kept as uploaded.

9. Predictive parser (predictive.c)

Table-driven predictive parser for an expression grammar using i as identifier and $ as end marker.

Source code (reproduced from uploaded file)

```
#include <stdio.h>

char stack[100];
char input[100];
int top = -1, ip = 0;

void push(char c)
{
    stack[++top] = c;
}

char pop()
{
    return stack[top--];
}

char peek()
{
    return stack[top];
}

void firstfollow()
{
    printf("First(E) = { (, id }\n");
    printf("First(Q) = { +, e }\n");
    printf("First(T) = { (, id }\n");
    printf("First(R) = { *, e }\n");
    printf("First(F) = { (, id }\n");
}

int col(char ch)
{
    switch (ch)
    {
    case 'i':
        return 0;
    case '+':
        return 1;
    case '*':
        return 2;
    case '(':
        return 3;
    case ')':
        return 4;
    case '$':
        return 5;
    }
    return -1;
}

int row(char ch)
{
    switch (ch)
    {
    case 'E':
        return 0;
    case 'Q':
        return 1;
    case 'T':
        return 2;
    case 'R':
        return 3;
    case 'F':
        return 4;
    }
    return -1;
}

char *table[5][6] =
    {
        {"TQ", "", "", "TQ", "", ""},
        {"", "+TQ", "", "", "#", "#"},
        {"FR", "", "", "FR", "", ""},
        {"", "#", "*FR", "", "#", "#"},
        {"i", "", "", "(E)", "", ""}};

int main()
{
    int j;

    printf("Enter input ending with $: ");
    scanf("%99s", input);

    push('$');
```

```
    push('E');

    while (peek() != '$')
    {
        char x = peek();
        char a = input[ip];

        if (x == a)
        {
            pop();
            ip++;
        }
        else if (x >= 'A' && x <= 'Z')
        {
            int r = row(x);
            int c = col(a);

            if (r == -1 || c == -1 || table[r][c][0] == '\0')
            {
                printf("Invalid string\n");
                return 0;
            }

            pop();

            if (table[r][c][0] != '#')
            {
                int len = 0;

                while (table[r][c][len] != '\0')
                    len++;

                for (j = len - 1; j >= 0; j--)
                    push(table[r][c][j]);
            }
        }
        else
        {
            printf("\nInvalid string\n");
            return 0;
        }
    }

    if (input[ip] == '$')
        printf("\nValid string");
    else
        printf("\nInvalid string");

    return 0;
}
```

Sample output

```
Enter input ending with $: i+i*i$
Valid string

Try invalid input:
Enter input ending with $: i+*i$
Invalid string
```

10. Recursive-descent expression parser (recursive.c)

Parses identifiers, parentheses, + and * using recursive-descent functions.

Source code (reproduced from uploaded file)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

char ip[100];
int pos = 0;

void E();
void Eprime();
void T();
void Tprime();
void F();

void error()
{
    printf("Syntax error at position %d\n", pos);
    exit(1);
}

void E()
{
    T();
    Eprime();
}

void Eprime()
{
    if (ip[pos] == '+')
    {
        pos++;
        T();
        Eprime();
    }
}

void T()
{
    F();
    Tprime();
}

void Tprime()
{
    if (ip[pos] == '*')
    {
        pos++;
        F();
        Tprime();
    }
}

void F()
{
    if (ip[pos] == '(')
    {
        pos++;
        E();

        if (ip[pos] == ')')
            pos++;
        else
            error();
    }
    else if (isalpha((unsigned char)ip[pos]))
    {
        pos++;

        while (isalnum((unsigned char)ip[pos]))
            pos++;
    }
    else
    {
        error();
    }
}

int main()
{
    printf("Enter a string: ");
    scanf("%99s", ip);

    E();
```

```
    if (ip[pos] == '\0')
        printf("String accepted\n");
    else
        error();

    return 0;
}
```

Sample output

```
Enter a string: a+b*c
String accepted

Try invalid input:
Enter a string: a+*b
Syntax error at position 2
```

11. Recursive-descent list parser (recursive2.c)

Parses a grammar containing a, parentheses, and comma-separated lists.

Source code (reproduced from uploaded file)

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

char ip[100];
int pos = 0;

void S();
void L();
void Lprime();

void error()
{
    printf("Syntax error and string not accepted!\n");
    exit(0);
}

void S()
{
    if (ip[pos] == '(')
    {
        pos++;
        L();

        if (ip[pos] == ')')
            pos++;
        else
            error();
    }
    else if (ip[pos] == 'a')
    {
        pos++;
    }
    else
    {
        error();
    }
}

void L()
{
    S();
    Lprime();
}

void Lprime()
{
    if (ip[pos] == ',')
    {
        pos++;
        S();
        Lprime();
    }
}

int main()
{
    printf("Enter string: ");
    scanf("%99s", ip);

    S();

    if (ip[pos] == '\0')
        printf("String is accepted\n");
    else
        error();

    return 0;
}
```

Sample output

```
Enter string: (a,a)
String is accepted

Try invalid input:
Enter string: (a,)
Syntax error and string not accepted!
```

12. SLR expression parser (slr1.c)

Shift-reduce parser for the grammar E→E+T | T, T→T*F | F, F→(E) | id.

SLR(1) ACTION table

ACTION[state, terminal] (s = shift, r = reduce, acc = accept)

State	id	+	*	(	)	$
0	s5			s4		
1		s6				acc
2		r2	s7		r2	r2
3		r4			r4	r4
4	s5			s4		
5		r6	r6		r6	r6
6	s5			s4		
7	s5			s4		
8		s6			s11	
9		r1	s7		r1	r1
10		r3			r3	r3
11		r5			r5	r5

GOTO[state, nonterminal]

State	E	T	F
0	1	2	3
1	—	—	—
2	—	—	—
3	—	—	—
4	8	2	3
5	—	—	—
6	—	9	3
7	—	—	10
8	—	—	—
9	—	—	—
10	—	—	—
11	—	—	—

Production numbering used by the program

r1: E → E + T

r2: E → T

r3: T → T * F

r4: T → F

r5: F → (E)

r6: F → id

Source code (reproduced from uploaded file)

```
#include <stdio.h>
#include <string.h>
```

```
#define MAX 100

int action[12][6] = {
    {5, 0, 0, 4, 0, 0},
    {0, 6, 0, 0, 0, 999},
    {0, -2, 7, 0, -2, -2},
    {0, -4, -4, 0, -4, -4},
    {5, 0, 0, 4, 0, 0},
    {0, -6, -6, 0, -6, -6},
    {5, 0, 0, 4, 0, 0},
    {5, 0, 0, 4, 0, 0},
    {0, 6, 0, 0, 11, 0},
    {0, -1, 7, 0, -1, -1},
    {0, -3, -3, 0, -3, -3},
    {0, -5, -5, 0, -5, -5}
};

int gotot[12][3] = {
    {1, 2, 3},
    {-1, -1, -1},
    {-1, -1, -1},
    {-1, -1, -1},
    {8, 2, 3},
    {-1, -1, -1},
    {-1, 9, 3},
    {-1, -1, 10},
    {-1, -1, -1},
    {-1, -1, -1},
    {-1, -1, -1},
    {-1, -1, -1}
};

int getcol(char *c)
{
    if (strcmp(c, "id") == 0)
        return 0;
    if (strcmp(c, "+") == 0)
        return 1;
    if (strcmp(c, "*") == 0)
        return 2;
    if (strcmp(c, "(") == 0)
        return 3;
    if (strcmp(c, ")") == 0)
        return 4;
    if (strcmp(c, "$") == 0)
        return 5;

    return -1;
}

int gotocol(char c)
{
    if (c == 'E')
        return 0;
    if (c == 'T')
        return 1;
    if (c == 'F')
        return 2;

    return -1;
}

void print(int st[], char symbol[], int top)
{
    int i;

    for (i = 0; i <= top; i++)
        printf("%c%d ", symbol[i], st[i]);
}

void printprod(int r)
{
    switch (r)
    {
    case 1:
        printf("Reduce E -> E+T");
        break;

    case 2:
        printf("Reduce E -> T");
        break;

    case 3:
        printf("Reduce T -> T*F");
        break;

    case 4:
        printf("Reduce T -> F");
        break;

    case 5:
        printf("Reduce F -> (E)");
```

```
        break;

    case 6:
        printf("Reduce F -> id");
        break;
    }
}

int main()
{
    char input[MAX][10];
    char symbol[MAX];
    int st[MAX];
    int n, i;
    int top = 0;

    printf("Enter number of tokens: ");
    scanf("%d", &n);

    if (n < 1 || n >= MAX)
    {
        printf("Invalid number of tokens!\n");
        return 1;
    }

    printf("Enter tokens (id, +, *, (, )):\n");

    for (i = 0; i < n; i++)
        scanf("%9s", input[i]);

    strcpy(input[n], "$");

    st[0] = 0;
    symbol[0] = '$';

    i = 0;

    printf("\n%-20s %-15s %s\n",
           "Stack", "Input", "Action");

    while (1)
    {
        int state = st[top];
        int col = getcol(input[i]);
        int act;

        if (col == -1)
        {
            printf("Invalid input\n");
            break;
        }

        act = action[state][col];

        print(st, symbol, top);
        printf("%-15s %-15s ", "", input[i]);

        if (act == 0)
        {
            printf("String Rejected\n");
            break;
        }

        if (act == 999)
        {
            printf("String Accepted\n");
            break;
        }

        if (act > 0)
        {
            printf("Shift %d\n", act);

            top++;
            st[top] = act;
            symbol[top] = input[i][0];

            i++;
        }
        else
        {
            int r = -act;
            int lhs;
            int rhslen;
            int pre;

            printprod(r);
            printf("\n");

            if (r == 1)
            {
                lhs = 0;
```

```
            rhslen = 3;
        }
        else if (r == 2)
        {
            lhs = 0;
            rhslen = 1;
        }
        else if (r == 3)
        {
            lhs = 1;
            rhslen = 3;
        }
        else if (r == 4)
        {
            lhs = 1;
            rhslen = 1;
        }
        else if (r == 5)
        {
            lhs = 2;
            rhslen = 3;
        }
        else
        {
            lhs = 2;
            rhslen = 1;
        }

        top = top - rhslen;

        pre = st[top];

        top++;
        st[top] = gotot[pre][lhs];

        if (lhs == 0)
            symbol[top] = 'E';
        else if (lhs == 1)
            symbol[top] = 'T';
        else
            symbol[top] = 'F';
    }
  }

  return 0;
}
```

Sample output

```
Enter number of tokens: 5
Enter tokens (id, +, *, (, )):
id
+
id
*
id

Stack               Input          Action
$0                  id             Shift 5
$0 i5               +              Reduce F -> id
$0 F3               +              Reduce T -> F
$0 T2               +              Reduce E -> T
$0 E1               +              Shift 6
$0 E1 +6            id             Shift 5
$0 E1 +6 i5         *              Reduce F -> id
$0 E1 +6 F3         *              Reduce T -> F
$0 E1 +6 T9         *              Shift 7
$0 E1 +6 T9 *7      id             Shift 5
$0 E1 +6 T9 *7 i5   $              Reduce F -> id
$0 E1 +6 T9 *7 F10  $              Reduce T -> T*F
$0 E1 +6 T9         $              Reduce E -> E+T
$0 E1               $              String Accepted
```

13. SLR parser for a/b grammar (slr2.c)

Shift-reduce parser for S→AB, A→a | ε, B→b | ε. The current program uses input ab$.

ACTION[state, terminal] (s = shift, r = reduce, acc = accept)

State	a	b	$
0	s3	r3	r3
1			acc
2		s5	r5
3		r2	r2
4			r1
5			r4

GOTO[state, nonterminal]

State	S	A	B
0	1	2	—
1	—	—	—
2	—	—	4
3	—	—	—
4	—	—	—
5	—	—	—

Production numbering used by the program

r1: S → AB

r2: A → a

r3: A → ε

r4: B → b

r5: B → ε

Source code (reproduced from uploaded file)

```
#include <stdio.h>
#include <string.h>

#define MAX 100

int action[6][3] = {
    {3, -3, -3},
    {0, 0, 999},
    {0, 5, -5},
    {0, -2, -2},
    {0, 0, -1},
    {0, 0, -4}};

int gotot[6][3] = {
    {1, 2, 0},
    {0, 0, 0},
    {0, 0, 4},
    {0, 0, 0},
    {0, 0, 0},
    {0, 0, 0}};

int getcol(char c)
{
    if (c == 'a')
        return 0;
    if (c == 'b')
        return 1;
    if (c == '$')
        return 2;

    return -1;
}
```

```
void print(int st[], int top)
{
    int i;

    for (i = 0; i <= top; i++)
        printf("%d", st[i]);
}

void printprod(int r)
{
    switch (r)
    {
    case 1:
        printf("Reduce S -> AB");
        break;

    case 2:
        printf("Reduce A -> a");
        break;

    case 3:
        printf("Reduce A -> epsilon");
        break;

    case 4:
        printf("Reduce B -> b");
        break;

    case 5:
        printf("Reduce B -> epsilon");
        break;
    }
}

int main()
{
    char in[MAX] = "ab$";
    int st[MAX];
    int top = 0;
    int ip = 0;

    st[0] = 0;

    printf("Input: %s\n\n", in);
    printf("%-20s %-15s %s\n",
           "Stack", "Input", "Action");

    while (1)
    {
        int state;
        int col;
        int act;

        state = st[top];
        col = getcol(in[ip]);

        if (col == -1)
        {
            printf("Invalid input\n");
            return 0;
        }

        act = action[state][col];

        print(st, top);
        printf("%-15s %-15s ",
               "", in + ip);

        if (act == 0)
        {
            printf("Error\n");
            break;
        }

        if (act == 999)
        {
            printf("String accepted\n");
            break;
        }

        if (act > 0)
        {
            printf("Shift %d\n", act);

            top++;
            st[top] = act;
            ip++;
        }
        else
        {
            int r = -act;
            int lhs;
```

```
            int rhslen;
            int pre;

            printprod(r);
            printf("\n");

            if (r == 1)
            {
                lhs = 0;
                rhslen = 2;
            }
            else if (r == 2)
            {
                lhs = 1;
                rhslen = 1;
            }
            else if (r == 3)
            {
                lhs = 1;
                rhslen = 0;
            }
            else if (r == 4)
            {
                lhs = 2;
                rhslen = 1;
            }
            else
            {
                lhs = 2;
                rhslen = 0;
            }

            top = top - rhslen;
            pre = st[top];

            top++;
            st[top] = gotot[pre][lhs];
        }
    }

    return 0;
}
```

Sample output

```
Input: ab$

Stack               Input           Action
0                   ab$             Shift 3
03                  b$              Reduce A -> a
02                  b$              Shift 5
025                 $               Reduce B -> b
024                 $               Reduce S -> AB
01                  $               String accepted
```

14. Dynamic stack (stack.c)

Implements push, pop, display and exit using dynamically allocated memory.

Source code (reproduced from uploaded file)

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *stack = NULL;
    int size, top = -1;
    int choice, value;

    printf("Enter the stack size: ");
    scanf("%d", &size);

    if (size <= 0)
    {
        printf("Invalid stack size!\n");
        return 1;
    }

    while (1)
    {
        printf("\n1. Push\n");
        printf("2. Pop\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
        case 1:
            printf("Enter the element: ");
            scanf("%d", &value);

            if (top == size - 1)
            {
                printf("Stack is full!\n");
                printf("Removing top element: %d\n",
                       stack[top]);

                // Remove the current top element
                top--;

                if (top == -1)
                {
                    free(stack);
                    stack = NULL;
                }
                else
                {
                    int *temp = realloc(
                        stack, top + 1);

                    if (temp != NULL)
                    {
                        stack = temp;
                    }
                }
            }

            // Allocate memory for the new element
            int *temp = realloc(
                stack, (top + 2) * sizeof(int));

            if (temp == NULL)
            {
                printf("Memory allocation failed!\n");
                break;
            }

            stack = temp;
            stack[++top] = value;

            printf("%d pushed into the stack.\n",
                   value);
            break;

        case 2:
            if (top == -1)
            {
                printf("Stack is empty!\n");
            }
            else
            {
                printf("Popped element: %d\n",
```

```
                       stack[top]);
                top--;

                if (top == -1)
                {
                    free(stack);
                    stack = NULL;
                }
                else
                {
                    int *temp = realloc(
                        stack, (top + 1) * sizeof(int));

                    if (temp != NULL)
                    {
                        stack = temp;
                    }
                }
            }
            break;

        case 3:
            if (top == -1)
            {
                printf("Stack is empty!\n");
            }
            else
            {
                printf("Stack elements are:\n");
                for (int i = top; i >= 0; i--)
                {
                    printf("%d\n", stack[i]);
                }
            }
            break;

        case 4:
            free(stack);
            printf("Memory released. Exiting...\n");
            return 0;

        default:
            printf("Invalid choice!\n");
        }
    }

    return 0;
}
```

Sample output

```
Enter the stack size: 3

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter the element: 10
10 pushed into the stack.

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter the element: 20
20 pushed into the stack.

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Stack elements are:
20
10

1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 2
Popped element: 20

1. Push
2. Pop
```

```
3. Display
4. Exit
Enter your choice: 4
Memory released. Exiting...
```